

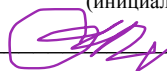
Правительство Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет «Высшая школа экономики»
Факультет компьютерных наук
Образовательная программа бакалавриата 09.03.04 «Программная инженерия»

ОТЧЕТ
по учебной (технологической) практике

в (на) Факультете компьютерных наук НИУ ВШЭ
(название структурного подразделения)

Выполнил студент
группы БПИ243
В. Е. Смирнов

(инициалы, фамилия)


(подпись)

Проверил:

Руководитель практики от факультета компьютерных наук

Гринберг Петр Маркович
(ФИО руководителя практики от ФКН)

Приглашенный преподаватель, Департамент больших данных и информационного поиска, ФКН НИУ ВШЭ
(должность руководителя практики от ФКН)

Дата _____

_____ (оценка)

_____ (подпись)

АННОТАЦИЯ

Отчет посвящен прохождению учебной (технологической) практики на Факультете компьютерных наук НИУ ВШЭ в формате обучения на мини-курсе «Основы глубинного обучения» в период с 01 июля 2026 г. по 14 июля 2026 г. Руководитель практики — Гринберг Петр Маркович, приглашенный преподаватель, Департамент больших данных и информационного поиска, ФКН НИУ ВШЭ.

В ходе практики изучены базовые и продвинутые темы глубинного обучения, включая обработку звука, а также выполнено итоговое практическое задание: разработана и обучена система противодействия голосовому спуфингу (Voice Anti-Spoofing / Countermeasure) на датасете ASVspoof 2019 Logical Access. В качестве архитектуры использована Light CNN (LCNN) со спектральным представлением сигнала на основе STFT. Обучение выполнялось в среде Kaggle с логированием экспериментов в Weights & Biases; код проекта разрабатывался локально на основе шаблона PyTorch Project Template.

По результатам официальной оценки на evaluation-разделе датасета достигнуто значение Equal Error Rate (EER) = 6,8113 %. Лучший результат получен на контрольной точке checkpoint-epoch9.pth. Репозиторий проекта и журнал экспериментов приведены в приложениях.

СОДЕРЖАНИЕ

Аннотация	2
1. Цель и задачи практики	4
2. Описание места прохождения практики	4
3. Программа мини-курса	5
4. Постановка задачи и обзор предметной области	5
5. Используемые материалы, методы и средства	6
6. Архитектура модели и экспериментальная постановка	7
7. Результаты экспериментов и анализ	8
8. Заключение	12
Список использованных источников	13
Рабочий план-график прохождения практики	15
Приложение А. Ссылки на репозиторий и журнал экспериментов	16
Приложение Б. Результаты оценки контрольных точек	17
Приложение В. Фрагменты реализации	18

1. Цель и задачи практики

Цель прохождения практики: изучить основы глубинного обучения и написания кода для нейронных сетей.

Задачи практики:

- изучить основы глубинного обучения;
- научиться обучать и использовать нейронные сети в коде;
- применить знания на практике путем решения домашнего задания.

В рамках третьей задачи было выполнено домашнее задание мини-курса: реализация системы Countermeasure для детектирования поддельной речи на Logical Access разделе датасета ASVspoof 2019 с ограничением на архитектуру LCNN и использованием STFT в качестве front-end.

2. Описание места прохождения практики

Практика проходила на Факультете компьютерных наук Национального исследовательского университета «Высшая школа экономики» в формате обучения на мини-курсе «Основы глубинного обучения».

Организатор мини-курса и руководитель практики: Гринберг Петр Маркович, Приглашенный преподаватель, Департамент больших данных и информационного поиска, ФКН НИУ ВШЭ. Информация о мини-курсах ФКН доступна на странице центра практик и проектной работы: <https://cs.hse.ru/cpppr/mini-course26>.

Факультет компьютерных наук НИУ ВШЭ осуществляет подготовку специалистов в области программной инженерии, анализа данных и искусственного интеллекта. Мини-курсы ФКН используются как одна из допустимых форм прохождения учебной практики и позволяют совместить теоретическое освоение современных методов с выполнением прикладного проекта.

Период прохождения практики: с 01 июля 2026 г. по 14 июля 2026 г.

3. Программа мини-курса

Содержание мини-курса «Основы глубинного обучения» включало два крупных блока.

1. Основные концепции глубинного обучения. В рамках блока рассматривались принципы построения нейронных сетей, обучение с учителем, функции потерь, оптимизаторы, регуляризация, а также практика реализации пайплайна обучения в PyTorch.

2. Продвинутое темы: мультимодальные сети, большие модели, обработка звука. Отдельное внимание уделялось задачам, связанным с анализом аудиосигналов, что непосредственно использовалось при выполнении итогового задания по детектированию голосового спуфинга.

Итоговым элементом контроля являлось домашнее задание по построению Countermeasure-системы на ASVspoof 2019 LA. Требовалось реализовать LCNN, обучить модель, предоставить логи обучения (Weights & Biases), а также сформировать файл предсказаний для официальной оценки по метрике EER.

4. Постановка задачи и обзор предметной области

Современные системы автоматической верификации диктора (Automatic Speaker Verification, ASV) уязвимы к атакам спуфинга: синтезированная речь, преобразование голоса и воспроизведение записи могут быть использованы для обхода биометрической защиты. Поэтому наряду с самой ASV-системой необходимы отдельные средства противодействия — Countermeasure (CM) системы, которые классифицируют входной аудиосигнал как подлинный (bonafide) либо поддельный (spoof).

В качестве учебной и исследовательской площадки широко применяется датасет ASVspoof 2019. В настоящей работе использовалась Logical Access (LA) часть датасета, содержащая атаки на основе синтеза и преобразования речи. Задача формулируется как бинарная классификация высказываний.

Основной метрикой качества CM-систем в ASVspoof является Equal Error Rate (EER) — значение порога, при котором доля ложных принятий spoof-примеров (False Acceptance Rate) равна доле ложных отклонений bonafide-примеров (False Rejection Rate). Чем ниже EER, тем лучше система. В задании мини-курса EER измерялся на evaluation-разделе LA в процентах.

Согласно условию домашнего задания, решение ограничивалось архитектурой Light CNN (LCNN) по материалам работ STC и связанным recipe обучения. В качестве front-end рекомендовалось использовать STFT (в литературе также обозначается как FFT-спектрограмма). Реализации, доступные в открытых репозиториях, использовать было нельзя: необходимо было самостоятельно воспроизвести архитектуру и пайплайн обучения.

5. Используемые материалы, методы и средства

Для выполнения задания применялись следующие источники и инструменты.

Датасет. ASVspoof 2019 Logical Access: протоколы train/dev/eval и соответствующие наборы аудиофайлов в формате FLAC. Данные подключались через Kaggle Dataset.

Программный каркас. Проект реализован как вариант PyTorch Project Template (Hydra-конфигурации, модульная структура, тренер, инференс, логирование). Это позволило отделить описание эксперимента от кода и обеспечить воспроизводимость запусков.

Библиотеки. Python, PyTorch, torchaudio, Hydra, Weights & Biases, Для официальной проверки предсказаний использовался предоставленный скрипт grading.py.

Среда выполнения. Код писался и отлаживался локально; полноценное обучение модели выполнялось на платформе Kaggle (GPU). Журнал экспериментов велся в Weights & Biases, итоговый отчет по run доступен по ссылке в приложении А.

Методические материалы. Изучались evaluation plan ASVspoof 2019, статья по Light CNN, работа STC по anti-spoofing, а также comparative study с обсуждением выбора функции потерь (Cross-Entropy vs A-Softmax).

6. Архитектура модели и экспериментальная постановка

6.1. Предобработка сигнала

Входной аудиосигнал преобразовывался в логарифмическую магнитудную спектрограмму STFT. Параметры front-end были зафиксированы следующим образом: $n_fft = 512$, $hop_length = 160$, $win_length = 400$, $max_frames = 600$. К спектрограмме добавлялся канал ($shape: batch \times 1 \times freq \times time$), после чего признаки поступали в сверточную сеть. Ограничение по числу кадров обеспечивало единообразие размера входа в батче: более длинные спектрограммы обрезались, более короткие дополнялись нулями.

6.2. Модель LCNN

Архитектура основана на Light CNN с модулями Max-Feature-Map (MFM). MFM реализует конкуренцию между картами признаков и позволяет уменьшить число активных каналов без введения дополнительных нелинейностей в духе классического ReLU. Сверточный backbone последовательно применяет MFM-свертки, MaxPooling и BatchNorm, после чего признаки вытягиваются в вектор и подаются в классификатор.

Классификатор содержит полносвязный MFM-слой (размерность 80), слой Dropout с вероятностью 0,75, BatchNorm1d и финальный линейный слой на два класса (spoof / bonafide). Dropout размещен перед финальным BatchNorm — в соответствии с указанием из условия задания. Dropout используется для регуляризации полносвязной части модели и снижения риска переобучения. Его расположение перед финальным BatchNorm соответствует требованиям задания. (датасет не сбалансирован)

6.3. Функция потерь и обоснование выбора

В работе использовалась функция потерь Cross-Entropy Loss. От применения A-Softmax было решено отказаться, поскольку данная функция потерь усложняет процесс обучения и требует более тщательной настройки модели. В рамках данной работы основной целью являлось построение стабильной базовой системы детектирования спуфинга, поэтому был выбран более простой и широко применяемый подход на основе Cross-Entropy Loss.

6.4. Гиперпараметры обучения

Обучение проводилось в течение 20 эпох оптимизатором Adam с начальной скоростью обучения $3 \cdot 10^{-4}$. Использовался планировщик StepLR с коэффициентом $\gamma = 0,9$; шаг планировщика совпадал с длиной эпохи (1586 итераций), что давало плавное ступенчатое уменьшение learning rate. Размер батча составил 16. Для воспроизводимости фиксировался $\text{seed} = 1$. В процессе обучения сохранялись контрольные точки; мониторинг качества на валидации вел к выбору `model_best`, однако итоговый выбор модели для сдачи выполнялся по официальной метрике EER на evaluation-наборе.

Выбор указанных гиперпараметров опирался на типовые recipe для LCNN в задачах anti-spoofing и на практическую устойчивость обучения: при заданном learning rate модель быстро снижала train loss, а StepLR позволял уточнять веса на поздних эпохах без резкой расходимости.

7. Результаты экспериментов и анализ

Ниже приведены графики обучения финального запуска (`final_run`), построенные по экспортированным логам Weights & Biases. Отдельно обсуждается официальная оценка контрольных точек на evaluation-разделе.

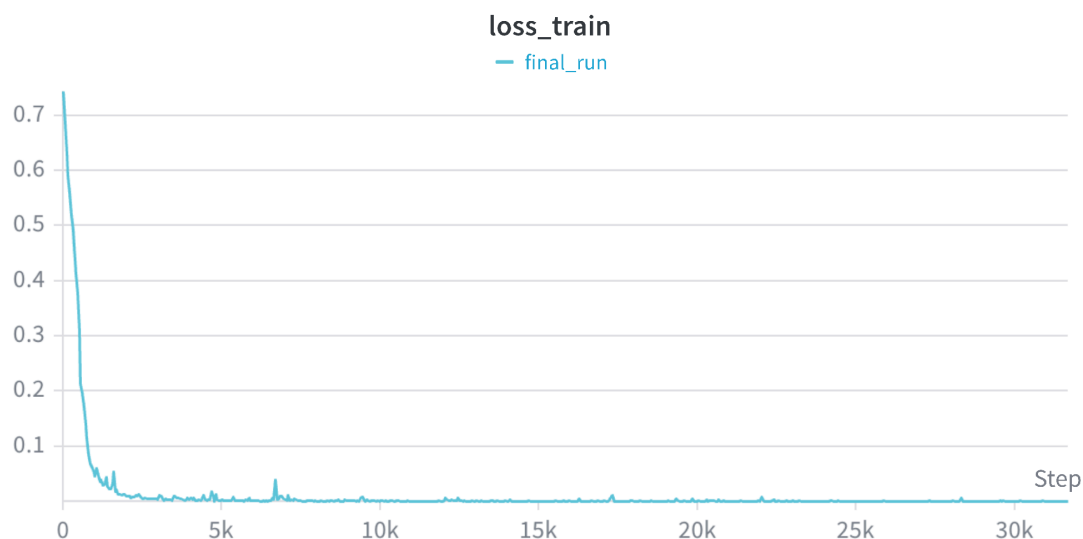


Рисунок 1 — Динамика функции потерь на обучающей выборке ($loss_{train}$)

На рисунке 1 видно, что значение функции потерь на обучении быстро убывает в первые тысячи шагов: от примерно 0,7–0,75 в начале запуска до величин порядка 10^{-2} уже к 2–5 тысячам итераций. Далее кривая выходит на плато вблизи нуля и остается стабильной до конца обучения (более 30 тысяч шагов). Такая динамика свидетельствует о том, что пайплайн обучения корректен, а выбранные архитектура и learning rate позволяют модели эффективно подстраиваться под признаки различия bonafide и spoof на train-разделе. Небольшие локальные всплески потери типичны для стохастической оптимизации и не нарушают общего тренда.

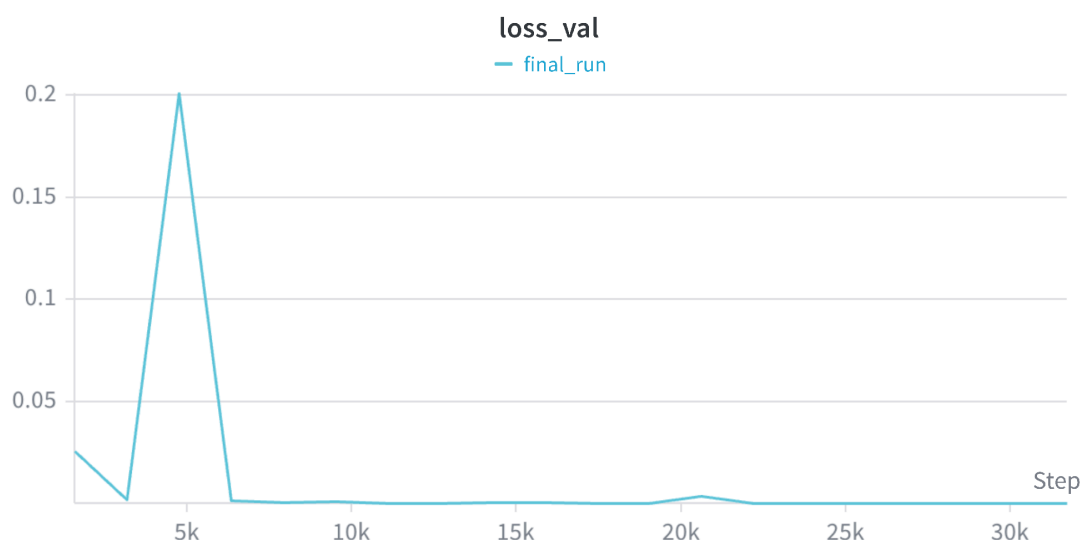


Рисунок 2 — Динамика функции потерь на валидационной выборке (*loss_val*)

Рисунок 2 показывает поведение validation loss. В начале обучения наблюдается выраженный пик около 4–5 тысяч шагов (значение порядка 0,2), после которого потеря резко снижается и стабилизируется на очень низком уровне. Ранний всплеск возможно интерпретировать как этап неустойчивой адаптации весов при еще относительно большом learning rate и изменении распределения предсказаний. Важно, что после восстановления кривая остается почти плоской: модель не демонстрирует монотонного роста валидационной потери, который был бы явным признаком сильного переобучения по Cross-Entropy. Вместе с тем низкий val loss не тождественен лучшему EER на официальном evaluation, поэтому контрольные точки дополнительно переоценивались внешним скриптом.

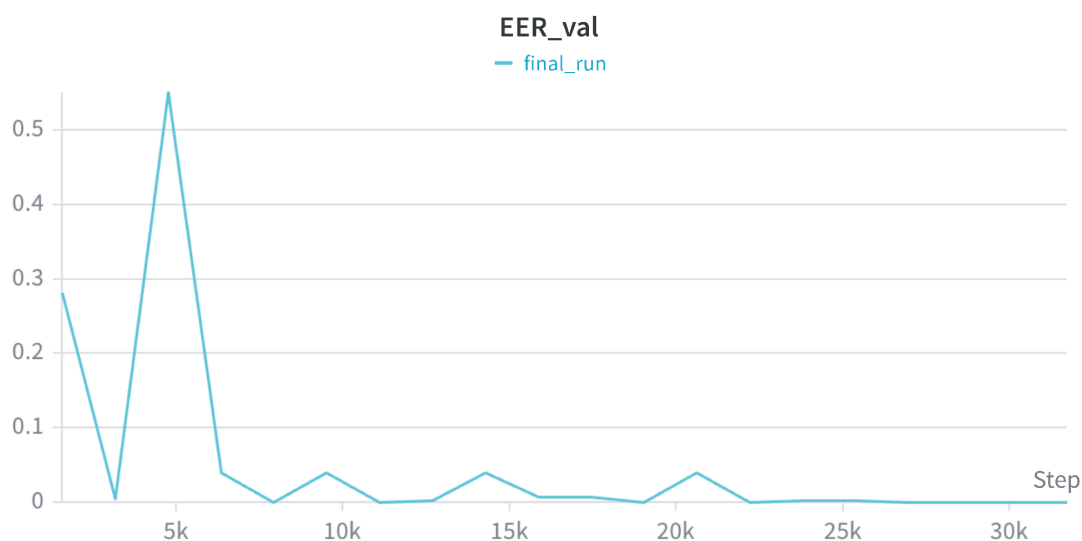


Рисунок 3 — Динамика Equal Error Rate на валидации (*EER_val*)

На рисунке 3 представлена динамика EER на валидационном разделе. После стартового значения около 0,28 метрика сначала резко улучшается, затем в районе 5 тысяч шагов наблюдается сильный выброс ($EER \approx 0,55$), синхронный с пиком validation loss. Далее EER возвращается к малым значениям и в целом колеблется вблизи нуля с отдельными локальными подъемами. К концу обучения валидационный EER стабилизируется. Такая картина означает, что по внутренней валидации модель выглядит очень сильной; однако официальная

оценка на evaluation-наборе дает более реалистичную величину ошибки обобщения, поэтому именно она использовалась для выбора сдаваемой контрольной точки.

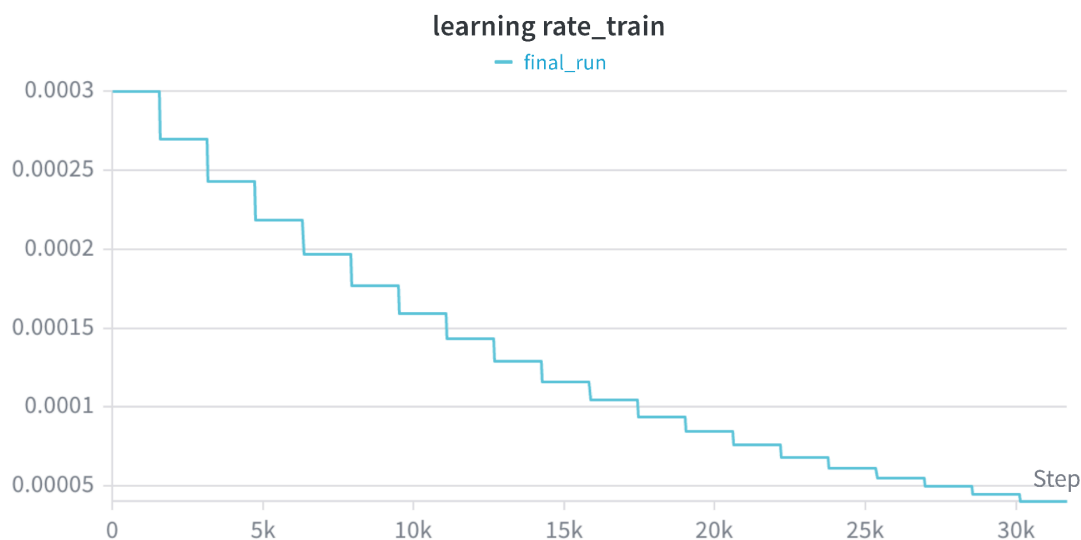


Рисунок 4 — Расписание скорости обучения (learning rate)

Рисунок 4 иллюстрирует ступенчатое уменьшение learning rate по расписанию StepLR. Обучение начинается с $3 \cdot 10^{-4}$; каждые 1586 шагов скорость умножается на 0,9, образуя «лестницу» спада. К концу запуска learning rate снижается примерно до $4 \cdot 10^{-5}$. Такая стратегия соответствует стандартной практике: сравнительно большой шаг в начале ускоряет поиск хорошей области параметров, а последующее уменьшение способствует более аккуратной донастройке. Сопоставление рисунка 4 с рисунками 1–3 показывает, что основная фаза падения потерь приходится на период относительно высокого learning rate, а дальнейшее обучение идет уже в режиме уточнения.

7.1. Официальная оценка и выбор контрольной точки

Контрольная точка model_best.pth автоматически выбиралась тренером по минимальному значению val_EER и на официальной evaluation-оценке показала EER = 12,3434 %, что совпадает с результатом последней эпохи. Поскольку итоговая оценка задания выполняется внешним скриптом на evaluation-разделе, были дополнительно проверены все сохраненные

checkpoint-файлы. Наилучший результат достигнут на контрольной точке 9-й эпохи: EER = 6,8113 %. Именно этот результат использован в итоговом файле предсказаний vesmirnov.csv.

Расхождение между моделью, выбранной по внутренней валидационной метрике, и контрольной точкой с наименьшим официальным EER может быть обусловлено различиями между dev- и eval-разделами, а также несовпадением внутренних и итоговых критериев оценки. В рамках анализа результатов были ретроспективно проверены сохранённые контрольные точки с использованием предоставленного скрипта оценивания. Наименьшее значение EER было получено для контрольной точки девятой эпохи.

Полная таблица официальных EER по контрольным точкам приведена в приложении Б. Ближайшие к лучшему результаты показали эпохи 17 (6,9866 %) и 11 (7,2342 %), тогда как ряд средних и поздних эпох существенно хуже. Это подтверждает, что качество СМ-системы в данной постановке чувствительно к моменту остановки обучения.

Достигнутый EER = 6,8113 % находится в целевом диапазоне задания (ниже 10,9 %) и соответствует работоспособной базовой LCNN-системе. Значения, близкие к лучшим результатам литературы по ASVspoof 2019, в рамках мини-курса не требовались: пороги оценивания были намеренно смягчены, чтобы сделать акцент на корректной реализации пайплайна.

8. Заключение

В период учебной практики на мини-курсе «Основы глубинного обучения» были изучены ключевые концепции deep learning и продвинутые темы, связанные в том числе с обработкой звука. Полученные знания применены при разработке системы детектирования голосового спуфинга на датасете ASVspoof 2019 LA.

Практическим результатом работы стала реализованная LCNN-модель со STFT front-end, обученная в воспроизводимом пайплайне на базе PyTorch

Project Template с логированием в Weights & Biases. Итоговое качество на официальной оценке составляет EER = 6,8113 % (checkpoint-epoch9.pth). Таким образом, цель и задачи практики выполнены: освоены основы обучения нейронных сетей, сформирован навык экспериментальной работы с аудиомodelью, успешно сдано домашнее задание мини-курса.

Основные сложности были связаны с необходимостью аккуратно воспроизвести архитектуру LCNN/MFM без опоры на готовые внешние реализации, настроить полный цикл train/val/inference и корректно интерпретировать разницу между внутренними метриками и официальным EER. Полученный опыт полезен для дальнейшей работы с задачами классификации сигналов и системами машинного обучения производственного уровня.

Результаты практики потенциально могут быть использованы при подготовке выпускной квалификационной работы в случае выбора тематики, связанной с анализом речи, обнаружением синтезированного аудио, биометрической безопасностью или построением воспроизводимых пайплайнов обучения нейросетевых моделей. Конкретная тема ВКР на момент составления отчета не фиксируется; речь идет лишь о возможном направлении дальнейшего применения накопленного опыта и программного задела.

Список использованных источников

1. ASVspoof 2019: Automatic Speaker Verification Spoofing and Countermeasures Challenge Evaluation Plan. URL: https://www.asvspoof.org/asvspoof2019/asvspoof2019_evaluation_plan.pdf (дата обращения: 26.07.2026).
2. Wu X., He R., Sun Z., Tan T. A Light CNN for Deep Face Representation with Noisy Labels // arXiv:1511.02683. 2018.
3. Lavrentyeva G., Novoselov S., Tseren A., Volkova M., Gorlanov A., Kozlov A. STC Antispoofing Systems for the ASVspoof2019 Challenge // arXiv:1904.05576. 2019.

4. Wang X. et al. ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech // Computer Speech & Language. 2020. Vol. 64.
5. Zhang Y., Jiang F., Duan Z. One-class learning towards generalized voice spoofing detection // arXiv:2103.11326. 2021.
6. Blinorot. PyTorch Project Template. URL: https://github.com/Blinorot/pytorch_project_template (дата обращения: 26.07.2026).
7. Факультет компьютерных наук НИУ ВШЭ. Мини-курсы. URL: <https://cs.hse.ru/cppr/mini-course26> (дата обращения: 26.07.2026).
8. Weights & Biases. Experiment tracking. URL: <https://wandb.ai> (дата обращения: 26.07.2026).
9. ASVspoof 2019 Dataset (Kaggle mirror). URL: <https://www.kaggle.com/datasets/awsaf49/asvspoof-2019-dataset> (дата обращения: 26.07.2026).
10. Репозиторий проекта Voice Anti-Spoofing. URL: <https://github.com/l42nal/voice-antispoofing> (дата обращения: 26.07.2026).

Рабочий план-график прохождения практики

Ниже приведен рабочий график прохождения практики с отметками о выполнении.

№ п/п	Сроки проведения	Планируемые работы	Отметка о выполнении
1	01.07.2026	Инструктаж по охране труда, технике безопасности, пожарной безопасности и правилам внутреннего трудового распорядка	выполнено
2	01.07.2026 — 06.07.2026	Изучение основных концепций глубинного обучения	выполнено
3	07.07.2026 — 09.07.2026	Знакомство с продвинутыми разделами (мультимодальные сети, большие модели, обработка звука)	выполнено
4	10.07.2026 — 14.07.2026	Решение домашнего задания: система Voice Anti-Spoofing на ASVspoof 2019 LA	выполнено
5	14.07.2026	Оформление индивидуального задания и отчета по практике	выполнено

Приложение А. Ссылки на репозиторий и журнал экспериментов

Исходный код проекта размещен в репозитории GitHub:

<https://github.com/l42nal/voice-antispoofing>

Журнал экспериментов (Weights & Biases Report):

https://wandb.ai/na4242na-hse-university/pytorch_template/reports/ASVspoof-2019-LA-LCNN-Training-Report--VmlldzoxNzU4MjU5MA?accessToken=mhjlfsabjbwqraaxtppx0abqfvdvdhfov20gwrz1mbe0g9yump2gxac7h0xh59ej

В репозитории также находятся файл итоговых предсказаний `results/vesmirnov.csv` и таблица `results/checkpoint_eer_results.csv` с официальными значениями EER для сохраненных контрольных точек.

Приложение Б. Результаты оценки контрольных точек

В таблице Б.1 приведены значения официального EER (%) для сохраненных checkpoint-файлов. Строки упорядочены по возрастанию EER.

Checkpoint	Эпоха	EER, %	Комментарий
checkpoint-epoch9.pth	9	6,8113	лучший результат
checkpoint-epoch17.pth	17	6,9866	
checkpoint-epoch11.pth	11	7,2342	
checkpoint-epoch4.pth	4	7,6247	
checkpoint-epoch18.pth	18	7,9538	
checkpoint-epoch12.pth	12	8,159	
checkpoint-epoch16.pth	16	8,3481	
checkpoint-epoch10.pth	10	8,5088	
checkpoint-epoch5.pth	5	8,5626	
checkpoint-epoch6.pth	6	8,7548	
checkpoint-epoch19.pth	19	9,1077	
checkpoint-epoch7.pth	7	9,2176	
checkpoint-epoch13.pth	13	10,39	
checkpoint-epoch1.pth	1	11,6215	
checkpoint-epoch2.pth	2	11,6792	
checkpoint-epoch15.pth	15	11,72	
checkpoint-epoch8.pth	8	11,7453	
checkpoint-epoch20.pth	20	12,3434	
model_best.pth	—	12,3434	автовыбор тренера
checkpoint-epoch14.pth	14	12,5626	
checkpoint-epoch3.pth	3	14,7536	

Таблица Б.1 — Официальный EER по контрольным точкам

Приложение В. Фрагменты реализации

Ниже приведены ключевые фрагменты реализации front-end и классификатора. Полный код доступен в репозитории (приложение А).

Фрагмент 1. Параметры STFT-преобразования:

```
STFTTransform(  
    n_fft=512,  
    hop_length=160,  
    win_length=400,  
    max_frames=600,  
)
```

Фрагмент 2. Классификатор LCNN (dropout перед BatchNorm):

```
nn.Sequential(  
    nn.Flatten(),  
    MFMLinear(in_features=18944, out_features=80),  
    nn.Dropout(p=0.75),  
    nn.BatchNorm1d(80),  
    nn.Linear(in_features=80, out_features=2),  
)
```

Фрагмент 3. Основные гиперпараметры обучения:

```
optimizer: Adam(lr=3e-4)  
lr_scheduler: StepLR(gamma=0.9, step_size=epoch_len)  
n_epochs: 20  
batch_size: 16  
loss: CrossEntropyLoss
```